

Project D: Dynamic Language

An interpreter with lexical closures

Krutaya Komanda

Ernest Kudakaev

Albert Arselgov

Nikita Khripunkov

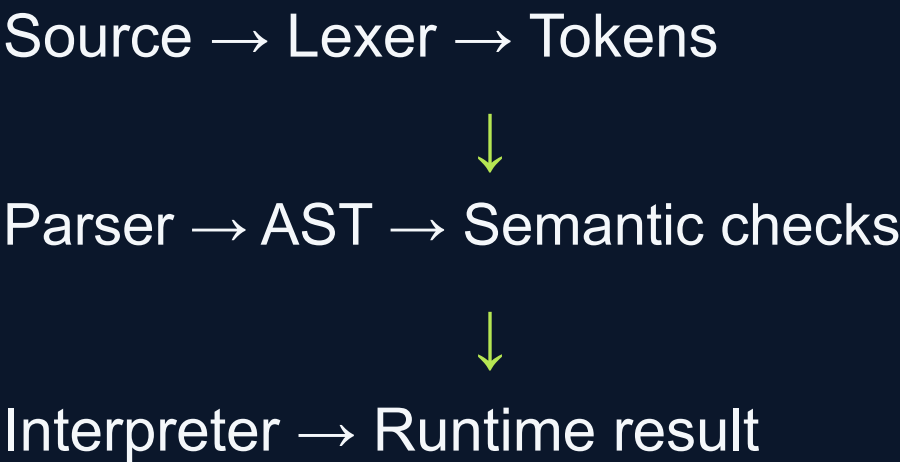
Language design and 14 sample programs

A Java interpreter built around an AST

Planned architecture: hand-written front end, direct execution of the AST.

Source language	Project D
Implementation	Java
Lexer	Hand-written
Parser	Hand-written recursive descent
Representation	AST
Target platform	Interpreter

EXECUTION PIPELINE



```
var x = 10 + 20;
```

Declaration(x) with Binary(+, 10, 20)

Variables do not have fixed types; values do.

Dynamic typing allows a variable to reference different kinds of values.

Primitive values

integer • real • boolean • string

```
var x = 10;  
x = "hello";  
x = true;
```

Composite values

array • tuple • function

Valid: assignment changes the value.

Statements: var, assignment, if / else, while, return, input / output

Explicit syntax, predictable runtime rules

Our choices keep the specification expressive and practical to implement.

Declarations & blocks `var` introduces names; braces and semicolons delimit code.

Conversions `integer → real`; `string + value` converts the value to text.

Collections Mutable heterogeneous arrays; immutable tuples.

Functions & scope First-class functions, lambdas and captured lexical environments.

Before execution

Invalid assignment target; return outside a function.

During execution

Type errors; writes to tuple elements.
Examples: `true + 10` and `"hello" - 10`.

Values and conversions

Three small programs define primitive values and conversion rules.

01 Literals

```
var name = "Alice";  
var age = 20;  
var height = 1.72;  
var student = true;  
print(name);  
print(age);  
print(height);  
print(student);
```

Expected output

Alice
20
1.72
true

02 Dynamic typing

```
var value = 10;  
print(value);  
value = "hello";  
print(value);  
value = true;  
print(value);
```

Expected output

10
hello
true

03 Conversions

```
var a = 10;  
var b = 2.5;  
var result = a + b;  
print(result);  
print("Result: " + result);
```

Expected output

12.5
Result: 12.5

Conditions and iteration

Independent programs demonstrate branching and repeated evaluation.

04 Conditions

```
var score = 75;  
if (score >= 60) {  
    print("passed");  
} else {  
    print("failed");  
}
```

Expected output
passed

05 Factorial loop

```
var n = 5;  
var result = 1;  
while (n > 1) {  
    result = result * n;  
    n = n - 1;  
}  
print(result);
```

Expected output
120

Control flow

if selects one branch; while repeats until its condition becomes false.

Arrays and tuples

Zero-based indexing works for both collections; only arrays allow element updates.

06 Mutable array

```
var numbers = [10, 20, 30];  
  
numbers[1] = 100;  
  
print(numbers[0]);  
print(numbers[1]);  
print(numbers[2]);
```

Expected output

10
100
30

07 Mixed array

```
var values = [  
    10, "hello", true, 3.14  
];  
  
print(values[0]);  
print(values[1]);  
print(values[2]);  
print(values[3]);
```

Expected output

10
hello
true
3.14

08 Tuple

```
var user = (  
    "Alex", 21, true  
);  
  
print(user[0]);  
print(user[1]);  
print(user[2]);
```

Expected output

Alex
21
true

Element updates are errors.

Named functions and recursion

Both programs are complete: each defines and calls its own function.

09 Named function

```
fun max(a, b) {  
  if (a > b) {  
    return a;  
  }  
  return b;  
}  
print(max(10, 20));
```

Expected output
20

10 Recursion

```
fun factorial(n) {  
  if (n <= 1) {  
    return 1;  
  }  
  return n * factorial(n - 1);  
}  
print(factorial(5));
```

Expected output
120

Function calls

Parameters receive values; return supplies the result to the caller.

Lambdas and lexical closures

Closure = function + captured lexical environment

11 Lambda

```
var square = lambda(x) => x * x;  
print(square(5));  
print(square(10));
```

Expected output

25
100

add10 captures x = 10

```
lambda(y) => x + y  
+ lexical environment: x = 10
```

12 Closure

```
fun makeAdder(x) {  
    return lambda(y) => x + y;  
}  
var add10 = makeAdder(10);  
var add100 = makeAdder(100);  
print(add10(5));  
print(add100(5));
```

Expected output

15
105

The captured x remains accessible after makeAdder returns.

Input, output and first-class functions

These examples complete the 14-program specification.

13 Input and output

```
print("Enter your name:");  
var name = input();  
print("Hello, " + name);
```

Input: Alice

Expected output
Enter your name:
Hello, Alice

14 Function variable

```
fun add(a, b) {  
    return a + b;  
}  
var operation = add;  
print(operation(4, 6));
```

Expected output
10

Next: implement the interpreter against these 14 examples.